

AI Weather API Setup & Configuration

Complete Installation, Infrastructure-as-Code (AWS CDK), Secrets
Provisioning, and Operating Procedures

+-----+ | AI WE

Document Ref: SOP-AIW-2026-V1

Deployment Target: AWS Lambda (Python
3.11)

Infrastructure: AWS CDK v2

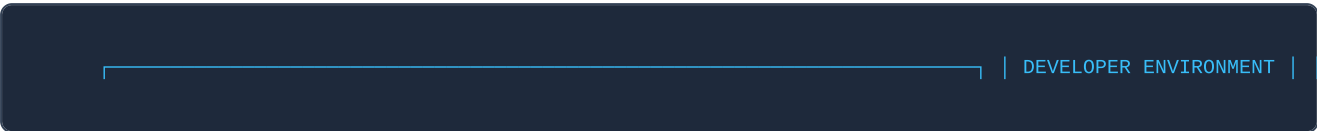
AI Engine: Google Gemini 2.5 Flash

1.1 SYSTEM PREREQUISITES

Before provisioning the AI Weather API, verify that the local engineering environment meets the following baseline toolchain dependencies:

Component	Required Version	Purpose / Usage
Python	3.11.x or higher	CDK application logic & Lambda runtime execution environment
Node.js	v18.x or v20.x LTS	AWS CDK CLI engine host environment
AWS CLI	v2.15.x+	Cloud credential authentication & regional resource control
Docker Desktop	4.25.x+	Automated containerized bundling of Python C-extensions for Lambda

1.2 ARCHITECTURE BLUEPRINT



Key Architecture Rule: Direct external API access from Lambda requires outbound HTTPS internet access via VPC NAT Gateway or default AWS non-VPC internet integration.

2.1 REPOSITORY INITIALIZATION

Execute the following shell commands to initialize the directory layout and isolate Python environment dependencies:

```
# Create main project workspace mkdir -p ai-weather-api/{stacks,lambda} cd ai-weather-api #
Initialize Python virtual environment python3 -m venv .venv # Activate virtual environment (macOS/
Linux) source .venv/bin/activate # Activate virtual environment (Windows PowerShell) # .venv\Scip
ts\Activate.ps1 # Upgrade core package managers pip install --upgrade pip setuptools wheel
```

2.2 INSTALLATION OF CDK & ROOT DEPENDENCIES

Install the AWS CDK framework CLI globally via Node Package Manager and install Python CDK construct libraries in the virtual environment:

```
# Install AWS CDK CLI globally npm install -g aws-cdk # Verify CDK CLI installation cdk --ve
rsion # Create root requirements.txt for CDK Infrastructure code cat << 'EOF' > requirements.txt a
ws-cdk-lib>=2.120.0 constructs>=10.0.0 aws-cdk.aws-lambda-python-alpha>=2.120.0a0 EOF # Install ro
ot dependencies pip install -r requirements.txt
```

Operational Note: Always verify that `.venv` is active before running `pip install` to avoid polluting the system Python runtime.

3.1 TARGET FILE STRUCTURE

Confirm the final directory structure matches the official production standard before proceeding to infrastructure coding:

```
ai-weather-api/ | ├── cdk.json # AWS CDK engine execution parameters |— app.py # CDK Application entry point
```

3.2 LAMBDA REQUIREMENTS DEFINITION

Define the isolated runtime packages required by the AWS Lambda function in `lambda/requirements.txt`:

```
cat << 'EOF' > lambda/requirements.txt httpx==0.27.0 google-genai>=0.1.1 EOF
```

3.3 CDK CONTEXT CONFIG (CDK.JSON)

Configure the CDK engine execution instructions in the root directory:

```
cat << 'EOF' > cdk.json { "app": "python app.py", "watch": { "include": ["**"], "exclude":  
["README.md", "cdk.out", ".venv"] }, "context": { "@aws-cdk/aws-lambda:recognizeLayerVersion": true } } EOF
```

4.1 THIRD-PARTY API KEY PROVISIONING

The AI Weather API relies on two primary external API credentials. Acquire these keys prior to stack deployment:

1. OpenWeatherMap API Key

Obtain a free/paid API key from openweathermap.org. Grants access to standard current weather endpoints (/data/2.5/weather).

2. Google Gemini API Key

Generate a key in Google AI Studio (aistudio.google.com). Enables access to the gemini-2.5-flash inference engine.

4.2 STORING SECRETS IN AWS SECRETS MANAGER (RECOMMENDED)

For production deployments, store secrets securely in AWS Secrets Manager using the AWS CLI rather than passing keys in plain text:

```
# Store OpenWeatherMap Key in AWS Secrets Manager aws secretsmanager create-secret \ --name "ai-weather/openweather-key" \ --description "API Key for OpenWeatherMap API" \ --secret-string "YOUR_OPENWEATHER_API_KEY" \ --region us-east-1 # Store Google Gemini Key in AWS Secrets Manager aws secretsmanager create-secret \ --name "ai-weather/gemini-key" \ --description "API Key for Google Gemini AI SDK" \ --secret-string "YOUR_GEMINI_API_KEY" \ --region us-east-1
```

Security Warning: Never commit plain-text API keys into Git repositories. Always use AWS Secrets Manager or CDK deployment context flags.

5.1 STACK CONSTRUCT (STACKS/AI_WEATHER_STACK.PY)

Implement the CDK infrastructure definition to provision Lambda and HTTP API Gateway resources:

```
import aws_cdk as cdk from constructs import Construct from aws_cdk import ( Stack, Duration,
aws_lambda as _lambda, aws_apigatewayv2 as apigw2, aws_apigatewayv2_integrations as integration
s, ) from aws_cdk.aws_lambda_python_alpha import PythonFunction class AiWeatherStack(Stack): def __
__init__(self, scope: Construct, construct_id: str, openweather_api_key: str, gemini_api_key: str,
**kwargs) -> None: super().__init__(scope, construct_id, **kwargs) # 1. Lambda function with autom
ated asset bundling weather_lambda = PythonFunction( self, "AiWeatherFunction", entry="lambda", in
dex="lambda_function.py", handler="lambda_handler", runtime=_lambda.Runtime.PYTHON_3_11, timeout=D
uration.seconds(10), memory_size=256, environment={ "OPENWEATHERMAP_API_KEY": openweather_api_key,
"GEMINI_API_KEY": gemini_api_key, }, ) # 2. HTTP API Gateway v2 http_api = apigw2.HttpApi( self,
"AiWeatherHttpApi", api_name="AI Weather Service API", cors_preflight=apigw2.CorsPreflightOptions(
allow_origins=["*"], allow_methods=[apigw2.CorsHttpMethod.GET], ), ) # 3. Add Lambda Route http_ap
i.add_routes( path="/weather", methods=[apigw2.HttpMethod.GET], integration=integrations.HttpLambd
aIntegration("WeatherIntegration", weather_lambda), ) cdk.CfnOutput(self, "ApiEndpoint", value=f"
{http_api.url}weather")
```

6.1 CDK APPLICATION ENTRY POINT (APP.PY)

Create the main `app.py` script that reads context parameters and initializes the CDK stack:

```
#!/usr/bin/env python3 import os import aws_cdk as cdk from stacks.ai_weather_stack import AiWeatherStack app = cdk.App() # Retrieve API keys passed via CLI context (-c openweatherApiKey=... -c geminiApiKey=...) openweather_api_key = app.node.try_get_context("openweatherApiKey") or "" gemini_api_key = app.node.try_get_context("geminiApiKey") or "" AiWeatherStack(app, "AiWeatherStack", openweather_api_key=openweather_api_key, gemini_api_key=gemini_api_key, env=cdk.Environment(account=os.getenv("CDK_DEFAULT_ACCOUNT"), region=os.getenv("CDK_DEFAULT_REGION", "us-east-1"))) app.synth()
```

6.2 AWS CDK BOOTSTRAPPING

Before deploying for the first time in an AWS account and region, bootstrap the CDK environment to create required S3 staging buckets:

```
# Retrieve active AWS Account ID AWS_ACCOUNT_ID=$(aws sts get-caller-identity --query "Account" --output text) # Execute CDK Bootstrap cdk bootstrap aws://${AWS_ACCOUNT_ID}/us-east-1
```

7.1 HANDLER IMPLEMENTATION (LAMBDA/LAMBDA_FUNCTION.PY)

Implement the core backend runtime handler that fetches raw metrics and generates the Gemini AI briefing:

```
import json import os import httpx from google import genai OWM_API_KEY = os.getenv("OPENWEATHERMAP_API_KEY") GEMINI_API_KEY = os.getenv("GEMINI_API_KEY") ai_client = genai.Client(api_key=GEMINI_API_KEY) if GEMINI_API_KEY else None def lambda_handler(event, context): query_params = event.get("queryStringParameters") or {} city = query_params.get("city") if not city: return build_response(400, {"error": "Query parameter 'city' is required"}) # Fetch weather metrics owm_url = f"http://api.openweathermap.org/data/2.5/weather?q={city}&appid={OWM_API_KEY}&units=metric" try: res = httpx.get(owm_url, timeout=5.0) if res.status_code != 200: return build_response(res.status_code, {"error": "Weather fetch failed", "details": res.json()}) data = res.json() except Exception as e: return build_response(500, {"error": str(e)}) temp = data["main"]["temp"] condition = data["weather"][0]["description"] humidity = data["main"]["humidity"] wind = data["wind"]["speed"] # Generate AI Briefing via Gemini ai_summary = "AI summary unavailable." if ai_client: try: prompt = f"Give a friendly 2-sentence weather briefing for {city}. Temp: {temp}°C, Condition: {condition}, Humidity: {humidity}%, Wind: {wind} m/s." ai_res = ai_client.models.generate_content(model="gemini-2.5-flash", contents=prompt) ai_summary = ai_res.text.strip() except Exception as err: ai_summary = f"Gemini error: {str(err)}" return build_response(200, {"city": city, "temperature_celsius": temp, "condition": condition, "humidity": humidity, "wind_speed_m_s": wind, "ai_summary": ai_summary }) def build_response(code, body): return {"statusCode": code, "headers": {"Content-Type": "application/json", "Access-Control-Allow-Origin": "*"}, "body": json.dumps(body)}
```

8.1 SYNTHESIZING CLOUDFORMATION TEMPLATES

Synthesize the CDK app to verify CloudFormation template generation prior to live deployment:

```
cdk synth \ -c openweatherApiKey="YOUR_OPENWEATHER_API_KEY" \ -c geminiApiKey="YOUR_GEMINI_API_KEY"
```

8.2 EXECUTING LIVE STACK DEPLOYMENT

Deploy the stack to your active AWS region. Docker must be running to handle Python package bundling:

```
cdk deploy \ -c openweatherApiKey="YOUR_OPENWEATHER_API_KEY" \ -c geminiApiKey="YOUR_GEMINI_API_KEY" \ --require-approval never
```

8.3 DEPLOYMENT OUTPUT VERIFICATION

Upon successful deployment, the CDK CLI will output the public HTTP endpoint URL:

```
===== Outputs: AiWeatherStack.ApiEndpoint
```

9.1 ENDPOINT VERIFICATION

Test the deployed live API endpoint using `curl` or any HTTP testing client:

```
# Test request for London weather
curl -X GET "https://a1b2c3d4.execute-api.us-east-1.amazonaws.com/weather?city=London"
# Expected JSON Response Output
{ "city": "London", "temperature_celsius": 14.5, "condition": "scattered clouds", "humidity": 72, "wind_speed_m_s": 4.1, "ai_summary": "London is experiencing mild temperatures around 14.5°C under scattered clouds. A lightweight sweater or jacket will be ideal for outdoor activities today." }
```

9.2 COMMON TROUBLESHOOTING MATRIX

Symptom	Root Cause	Resolution Procedure
400 Missing Parameter	Query missing ?city=	Pass required parameter e.g. ?city=Tokyo
500 API Key Missing	Context flags omitted in deployment	Re-deploy stack with -c openweatherApiKey=...
Docker Bundling Error	Docker service inactive locally	Start Docker Desktop before running cdk deploy

9.3 STACK TEARDOWN & DE-PROVISIONING

```
# Destroy all provisioned AWS Cloud resources
cdk destroy -c openweatherApiKey="dummy" -c ge
miniApiKey="dummy"
```